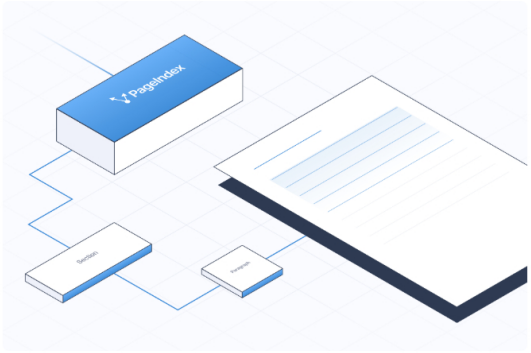
PageIndex: Next-Generation Vectorless, Reasoning-based RAG

Sep 19, 2025

Mingtian Zhang Yu Tang



GitHub repo Try PageIndex

Large Language Models (LLMs) have become powerful engines for document understanding and question answering. However, they are constrained by a fundamental architectural limit: the **context window**, i.e., the maximum number of tokens the model can process at once. Even though recent models support longer contexts, research such as Chroma's *context rot* study has shown that model performance deteriorates as context length grows. This makes it challenging for LLMs to accurately interpret and reason over long, complex, domain-specific documents such as financial reports or legal filings.

To mitigate this limitation, **Retrieval Augmented Generation (RAG)** has emerged as the dominant solution. Instead of passing the entire document into the model, RAG retrieves and feeds only the most relevant text chunks based on the user's query, thereby optimizing the effective context length. However, conventional **vector-based RAG** methods depend heavily on static semantic similarity and face several key limitations. To address these challenges, we introduce **PageIndex**, a **reasoning-based retrieval framework** that enables LLMs to dynamically navigate document structures and infer which sections are genuinely relevant, rather than merely retrieving text that appears semantically similar.

The Limitations of Vector-based RAG

Vector-based RAG relies on **semantic embeddings** and **vector databases** to identify relevant text chunks. In the *preprocessing stage*, the document is first *split* into smaller chunks, then each chunk is *embedded* into a vector space using an embedding model, and the resulting vectors are *stored* in a vector database such as Chroma or Pinecone. During the *query stage*, the user query is *embedded* using the same embedding model, the vector database is *searched* for semantically similar chunks, and the system *retrieves* the top-k results, which are then used to form the model's input context. While simple and effective for short texts, vector-based RAG faces several major challenges:

1. **Query and knowledge space mismatch**. Vector retrieval assumes that the *most semantically similar* text to the query is also the *most relevant*. But this isn't always true: queries often express *intent*, not *content*.

2. **Semantic similarity is not equivalent to relevance**. This is especially problematic in domain-specific documents (e.g., financial filings, legal documents, and technical manuals), where many passages share near-identical semantics but differ critically in relevance.

3. **Hard chunking breaks semantic and contextual integrity**. Documents are split into fixed-size chunks (e.g., 512 or 1000 tokens) for embedding. This "hard chunking" often cuts through sentences, paragraphs, or sections, fragmenting meaning and context.

4. **Cannot integrate chat history**. Each query is treated independently. The retriever doesn't know what's been asked or answered before.

5. **Hard to deal with in-document reference**. Documents often contain references such as "see Appendix G" or "refer to Table 5.3". Since these references don't share semantic similarity with the referenced content, traditional RAG misses them unless additional preprocessing (like a knowledge graph) is performed.

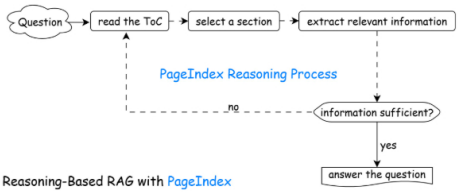
Because of these limitations, even advanced systems like **Claude Code** have moved away from traditional **vector-based RAG** for code retrieval, achieving superior precision and speed without relying on vector databases (see this blog post). We believe the same principle applies to **document retrieval**: rather than depending on static embeddings and semantic similarity, LLMs can reason over a structured representation of a document, deciding *where* to look next, not merely *what* looks similar. To this end, we introduce **PageIndex**, a **reasoning-based RAG framework** that overcomes the constraints of vector-based systems and brings the power of agentic retrieval to long-form, structured documents.

PageIndex: Reasoning-based Retrieval

PageIndex's Reasoning-based RAG mimics how humans naturally navigate and extract information from long documents. Unlike traditional vector-based methods that rely on static semantic similarity, this approach uses a *dynamic, iterative reasoning process* to actively decide where to look next based on the evolving context of the question.

It follows the iterative process below:

1. **Read the Table of Contents (ToC)**. Understand the document's structure and identify sections that might be relevant.
2. **Select a Section**. Choose the section most likely to contain useful information based on the question.
3. **Extract Relevant Information**. Parse the selected section to gather any content that could help answer the question.
4. **Is the Information Sufficient?**
    - **Yes** ⟶ Proceed to **Answer the Question**.
    - **No** ⟶ Return to **Step 1** and repeat the loop with another section.
5. **Answer the Question**. Once enough information is collected, generate a complete and well-supported answer.



In this process, the ToC serves as a key index for the document, enabling the LLM to navigate and retrieve information efficiently. We discuss how to design an LLM-friendly ToC in the next section.

"Table of Contents" Index for LLMs

We introduce a **JSON-based hierarchical structure** to represent a **Table of Contents (ToC)** for unstructured documents. The ToC acts as an **index tree** that organizes content into hierarchical nodes. Each node represents a logical section (e.g., chapter, paragraph, page) and may contain metadata, a description, and links to its sub-sections.

This approach allows an LLM to:

- Traverse structured content recursively.
- Retrieve targeted raw data by `node_id`.
- Associate contextual metadata (e.g., source type, topic, or semantic tags).

PageIndex Tree Index Example (JSON Format)

```
Node {
  node_id: string,         // Unique node identifier
  name: string,            // Human-readable label or title
  description: string,     // Optional detailed explanation of the node
  metadata: object,        // Arbitrary key-value pairs for context or attributes
  sub_nodes: [Node]        // Array of child nodes (recursive structure)
}
```

Notes:

- The `node_id` serves as a reference key to locate the corresponding raw data.
- The `sub_nodes` field allows recursive nesting, forming a full ToC tree.
- The `metadata` field can store semantic information such as document type, author, timestamp, or relevance scores.

Example PageIndex Tree:

```
...
{
  "node_id": "0006",
  "title": "Financial Stability",
  "start_index": 21,
  "end_index": 22,
  "summary": "The Federal Reserve ...",
  "sub_nodes": [
    {
      "node_id": "0007",
      "title": "Monitoring Financial Vulnerabilities",
      "start_index": 22,
      "end_index": 28,
      "summary": "The Federal Reserve's monitoring ..."
    },
    {
      "node_id": "0008",
      "title": "Domestic and International Cooperation and Coordination",
      "start_index": 28,
      "end_index": 31,
      "summary": "In 2023, the Federal Reserve collaborated ..."
    }
  ]
}
...
```

Each node in the ToC links directly to its corresponding **raw content** (e.g., text,

images, tables):

```
node_id → node_content (raw content, extracted text, images, etc.)
```

This mapping enables the LLM to **select and retrieve** specific nodes as needed, facilitating precise and context-aware information access.

Unlike a **vector database**, which stores an external, static embeddings index, the **JSON-based ToC index** resides *within* the LLM's *active reasoning context*. We call this an **in-context index** — a structure the model can directly reference, navigate, and reason over during inference. By integrating the index into the model's *context window*, the LLM can dynamically decide *where to look next* rather than depending solely on precomputed similarity scores. This enables in-context **reasoning-driven retrieval**, effectively addressing many of the constraints inherent in traditional vector-based RAG systems.

Overcoming the Limitations

1. *Query–Knowledge Space Mismatch*

Instead of relying solely on embedding similarity search, the LLM **uses reasoning to infer which section is likely to contain the answer**. It can "think" about document structure: e.g., "*Debt trends are usually in the financial summary section or Appendix G — let's look there.*" This dynamic inference bridges the gap between *query meaning* and *information location*.

2. *Semantic Similarity ≠ True Relevance*

Reasoning-based retrieval emphasizes **contextual relevance**, not just similarity. The model reads the Table of Contents (ToC) or PageIndex structure, interprets the query's intent, and **navigates** to sections that actually contain the answer, even if their language is different. This mirrors how humans find answers: by *understanding* the question, not just matching words.

3. *Hard Chunking Breaks Semantic Integrity*

Rather than chunking arbitrarily, reasoning-based RAG retrieves **semantically coherent sections** (e.g., full pages, sections, or chapters). If the model detects that a section is incomplete, it **iteratively fetches neighboring sections** (e.g., next page or sub-node) until context is sufficient. This preserves logical continuity and minimizes hallucination.

4. *Inability to Integrate Chat History*

Retrieval is **context-aware**: the model uses prior conversation history to refine its understanding of the current question. For instance, if the user previously asked about "financial assets," and now asks, "What about liabilities?", the retriever knows to explore *the same report section* under liabilities. This enables coherent, multi-step exploration across multiple turns.

5. *Poor Handling of In-Document References*

By leveraging the **PageIndex** or **ToC-based hierarchical structure**, reasoning-based retrieval can *follow references like a human reader*. When it encounters a phrase like "*see Appendix G*", the LLM navigates the index tree to that section and retrieves the relevant data. This allows accurate cross-referencing without manual link-building.

In the PageIndex MCP example, the query asked for the *total value of deferred assets*. The main section (pages 75–82) only reported the *increase* in value, not the total. On page 77, the text read:

> "Table 5.3 summarizes the income, expenses, and distributions of the Reserve Banks for 2023 and 2022. Appendix G of this report, 'Statistical Tables,' provides more detailed information…"

The reasoning-based retriever followed this cue to Appendix G, found the correct table, and returned the total deferred asset value — a task the vector-based retriever would likely fail.

Summary: Vector vs. Reasoning-based RAG

Limitation	Vector-based RAG	Reasoning-based RAG
1. Query–Knowledge Mismatch	Matches surface-level similarity; often misses true context	Uses inference to identify the most relevant document sections
2. Similarity ≠ Relevance	Retrieves semantically similar but irrelevant chunks	Retrieves contextually relevant information
3. Hard Chunking	Fixed-length chunks fragment meaning	Retrieves coherent sections dynamically
4. No Chat Context	Each query is isolated	Multi-turn reasoning considers prior context
5. Cross-References	Fails to follow internal document links	Follows in-text references via ToC/PageIndex reasoning

Conclusion

Vector-based RAG searches for similar text whereas reasoning-based RAG *thinks* about *where* to look and *why*. By combining structured document representations (like ToC Trees) with iterative reasoning, reasoning-based RAG enables LLMs to retrieve the **relevant information**, not just **similar information**, paving the way for a new generation of intelligent document understanding systems.

Check out our GitHub repo for open-source code, and cookbooks and tutorials for

additional usage guides and examples. The PageIndex service is available as a ChatGPT-style chat platform, or could be integrated via MCP or API.

Try PageIndex Now.

Citation

Please cite this work as:

```
Mingtian Zhang, Yu Tang and PageIndex Team,
"PageIndex: Next-Generation Vectorless, Reasoning-based RAG",
PageIndex Blog, Sep 2025.
```

Or use the BibTeX citation:

```
@article{zhang2025pageindex,
  author = {Mingtian Zhang and Yu Tang and PageIndex Team},
  title = {PageIndex: Next-Generation Vectorless, Reasoning-based RAG},
  journal = {PageIndex Blog},
  year = {2025},
  month = {September},
  note = {https://pageindex.ai/blog/pageindex-intro},
}
```

Back to Blog